ANEXO 01

DIAGRAMAS ENTIDAD-RELACIÓN DE LA BASE DE DATOS

Plataforma de Gestión de Información de Privacidad (PIMS) — GESTITEL E.I.R.L.

Documentación técnica de base de datos — Documento 01

Versión del sistema documentada: migraciones core/0001 – core/0043 · customer/0001 – customer/0004

Arequipa, Perú — Octubre 2026

TABLA DE CONTENIDO

A quién va dirigido 3
Qué contiene 3
Lo primero que hay que entender: no hay una sola base de datos por tabla 3
Convenciones de los diagramas 4
1. Diagrama general — bloques funcionales 5
2. Plataforma y empresas — esquema public 5
3. Gestión Documental 7
4. Procesos e Inventario de Activos 8
5. Metodología — matriz de evaluación y franjas de tasación 9
6. Matriz de Brecha y Anexo A (Declaración de Aplicabilidad) 10
7. Riesgos (DPIA / DPIIA) — el dominio más extenso 11
8. Consentimientos de titulares 13
9. Incidentes de seguridad y privacidad 14
10. Solicitudes ARCO-P (Mesa de Partes Digital) 16
11. Auditorías 18
12. Capacitaciones 19
13. Configuración del Sistema, identidad y bitácora 20
14. Dashboard Ejecutivo y EIPD (Datos Personales) 21
Resumen de cardinalidades singulares 23
Documentos relacionados 23

A quién va dirigido

A cualquier persona que necesite **entender de un vistazo cómo se organiza la información** de Gestitel: personal técnico que integra, migra o audita el sistema; responsables de protección de datos; auditores de un Sistema de Gestión de Seguridad de la Información.

No se requiere conocimiento previo del sistema. Si solo va a leer un documento de los cinco, lea este: los demás detallan lo que aquí se dibuja.

Qué contiene

Catorce diagramas: uno **general** que muestra los bloques funcionales y cómo se conectan entre sí, y trece **por dominio funcional**, cada uno con sus tablas, las columnas más representativas y las relaciones entre ellas.

Los diagramas se presentan como imágenes generadas a partir de su definición en **Mermaid**, un formato de texto que se conserva junto al código y que GitHub, GitLab, Visual Studio Code y la mayoría de visores de Markdown renderizan como imagen automáticamente. Si su visor no lo hace, el texto sigue siendo legible tal cual.

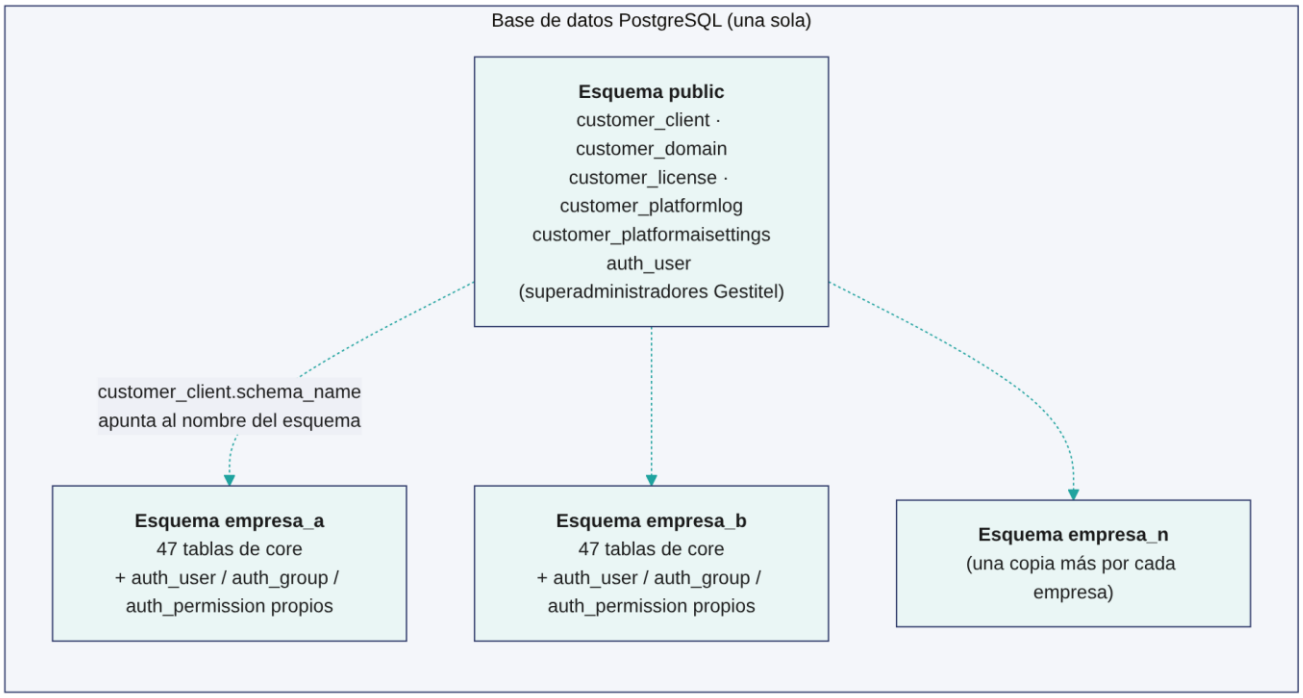
Lo primero que hay que entender: no hay una sola base de datos por tabla

Gestitel es una plataforma **multiempresa con aislamiento por esquema de PostgreSQL** (implementado con la librería `django-tenants`). Esto condiciona todo lo demás:

- Existe **un solo servidor y una sola base de datos** PostgreSQL.
- Dentro de ella hay **un esquema llamado `public`** que contiene el control de la plataforma: el catálogo de empresas, sus dominios, sus licencias y la bitácora de los superadministradores de Gestitel.
- Y hay **un esquema adicional por cada empresa cliente** (`empresa_a`, `empresa_b`, …). Cada uno de esos esquemas contiene **una copia completa** de las 47 tablas de negocio, más sus propias tablas de usuarios, grupos y permisos de Django.

Es decir: **`core_asset` no es una tabla, son N tablas** con el mismo nombre, una por empresa, cada una dentro de su esquema. Una consulta jamás puede devolver datos de dos empresas a la vez porque la conexión se sitúa en un único esquema antes de ejecutarla.

Ese es el mecanismo de aislamiento de datos de la plataforma. **Todos los diagramas de dominio de este documento describen el contenido de UN esquema de empresa.** El único que describe el esquema `public` es el diagrama 2.



> **Nota sobre las llaves foráneas entre esquemas: no existen**. `customer_client` no tiene una llave foránea hacia las tablas de una empresa, ni al revés. El vínculo es el texto de la columna `customer_client.schema_name`, que nombra el esquema. Esa separación es deliberada: elimina cualquier camino físico por el que una consulta pueda cruzar datos de dos empresas.

Convenciones de los diagramas

- `A ||--o{ B` — **uno a muchos**: una fila de A puede tener cero, una o muchas filas de B.
- `A ||--|| B` — **uno a uno**.
- `A }o--|| B` — **muchos a uno**, con la parte «muchos» opcional.
- `A }o--o{ B` — **muchos a muchos** (se materializa en una tabla puente).
- `PK` / `FK` / `UK` — llave primaria / llave foránea / valor único.

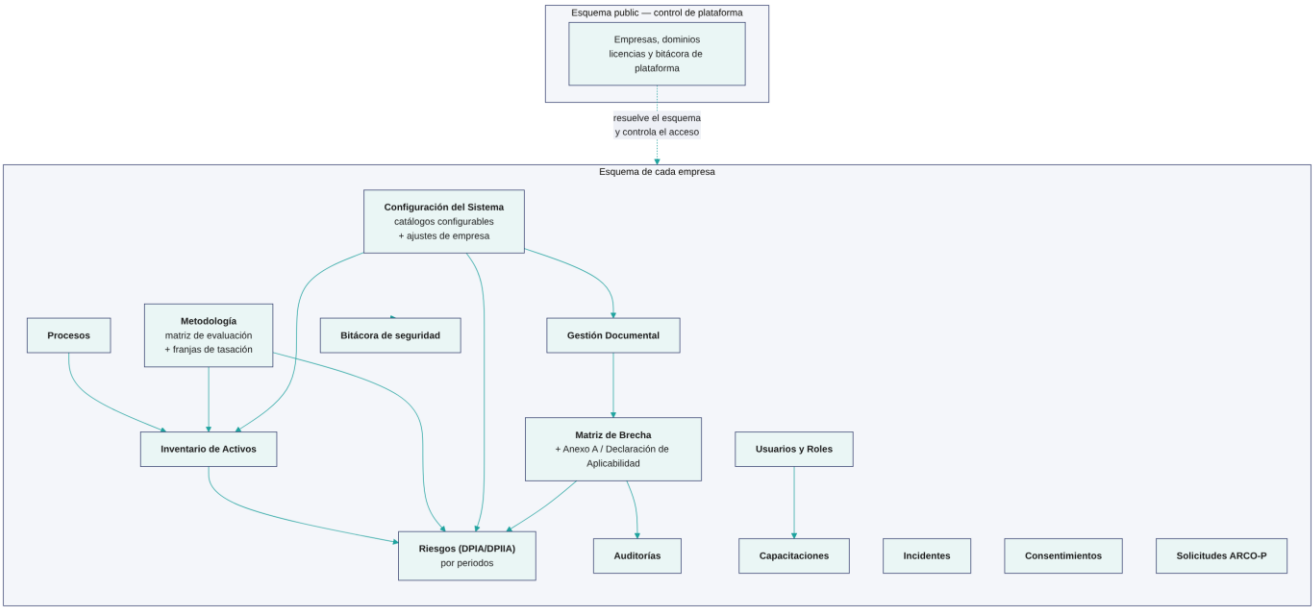
Las etiquetas de cada relación indican, entre paréntesis, la **regla de borrado**:

- **`CASCADE`** — al borrar el padre se borran los hijos. Se usa en las tablas hijas que no tienen sentido por separado (las versiones de un documento, las acciones de un incidente).
- **`PROTECT`** — impide borrar el padre mientras existan hijos. Protege los datos que son evidencia (un activo referenciado por un riesgo, un periodo con riesgos dentro).
- **`SET_NULL`** — al borrar el padre, la referencia del hijo queda vacía pero el hijo sobrevive. Se usa cuando la referencia es contexto, no sustancia (el usuario que subió un archivo, el requisito del que se copió un ítem de auditoría).

Por brevedad, los diagramas de dominio **omiten las siete columnas de auditoría** (`created_at`, `created_by_id`, `updated_at`, `updated_by_id`, `is_deleted`, `deleted_at`, `deleted_by_id`) que comparten casi todas las tablas de negocio, y muestran solo las columnas representativas de cada entidad. El detalle completo está en el documento **02 — Diccionario de datos**.

1. Diagrama general — bloques funcionales

Este diagrama **no muestra tablas**, sino los doce dominios funcionales del sistema y las dependencias reales entre ellos. Las flechas indican qué dominio consume datos de cuál.

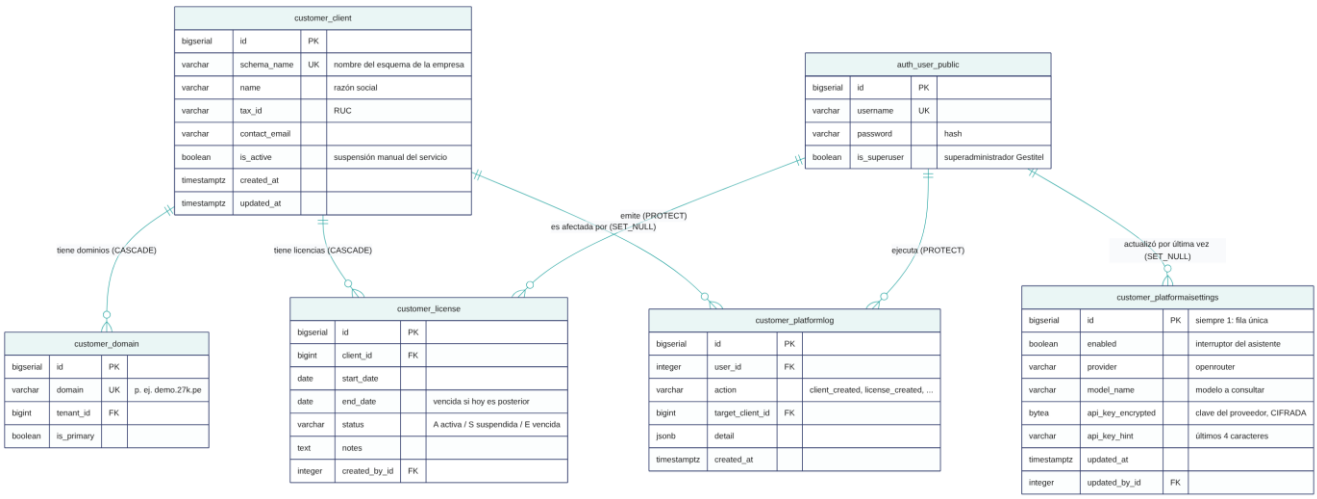


Cómo leer las dependencias principales:

- Un **activo** pertenece a un **proceso** y su valor se compara con las **franjas de tasación** de Metodología; de esa comparación depende si el activo entra al módulo de **Riesgos**.
- Un **riesgo** afecta a uno o más **activos**, se evalúa con la escala de la **matriz de evaluación** activa, propone controles del **Anexo A** declarados aplicables en la **Matriz de Brecha**, y pertenece siempre a un **periodo** de evaluación.
- Una **auditoría** importa su lista de verificación desde la **Matriz de Brecha**.
- Varios campos de lista (categoría de activo, categoría de documento, tipo de control, amenazas, vulnerabilidades) no tienen valores fijos en el código: se administran desde **Configuración del Sistema**.
- **Todos** los módulos escriben en la **bitácora de seguridad**.
- El **Dashboard Ejecutivo** (diagrama 14) no aparece como bloque propio en este diagrama de dependencias porque no introduce ninguna relación nueva entre dominios: lee en tiempo real de todos los bloques existentes y solo persiste dos tablas propias (metas y foto mensual), sin llave foránea hacia ninguno de ellos.
- La **EIPD** (diagrama 14) sí depende de un dominio existente: cada evaluación de impacto pertenece a exactamente un **activo** del Inventario (`INV`), igual que un riesgo.

2. Plataforma y empresas — esquema public

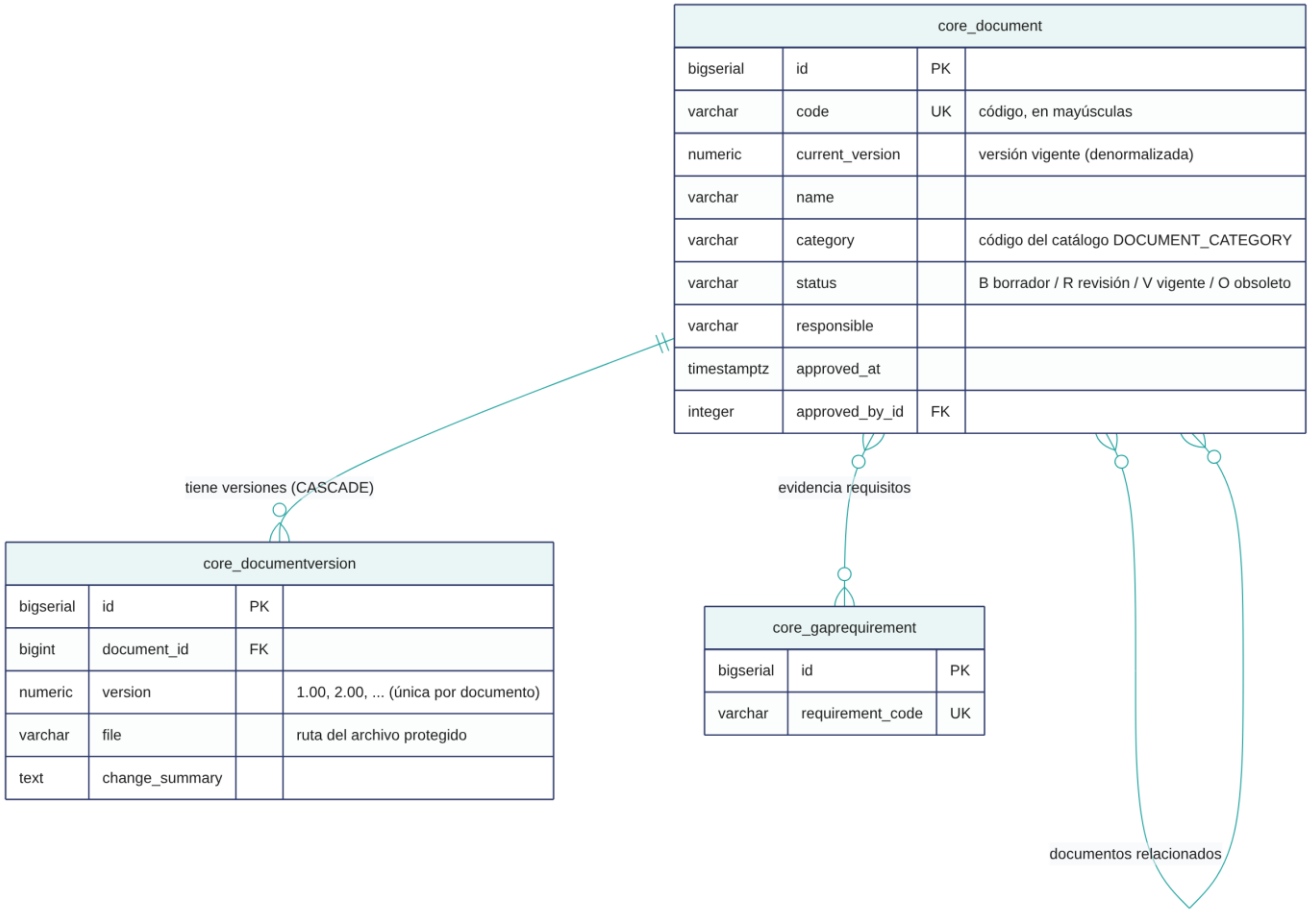
Es el único diagrama que describe el esquema `public`. Aquí viven los superadministradores de Gestitel y el registro de las empresas cliente.



Notas de lectura

- `customer_license` es un **historial inmutable por acumulación**: renovar una licencia no modifica la fila existente, crea una nueva. La licencia vigente es la de mayor `end_date`.
- `customer_client.is_active = false` (suspensión manual) y una licencia vencida son **dos bloqueos distintos** con mensajes distintos para el usuario.
- `auth_user_public` es la tabla estándar `auth_user` **dentro del esquema `public`**. Se dibuja con otro nombre para distinguirla de la tabla `auth_user` que cada empresa tiene en su propio esquema, que es una tabla diferente con usuarios diferentes.
- `customer_platformaisettings` es una **tabla de fila única** (su `id` vale siempre 1 y el borrado está bloqueado desde la aplicación): guarda la configuración **global** del asistente de consultas teóricas —encendido, proveedor, modelo y la clave de acceso al proveedor—. **No se relaciona con `customer_client`**: es una sola configuración para toda la plataforma, que administra Gestitel; ninguna empresa cliente la ve ni la modifica. La clave **no se guarda en claro**: `api_key_encrypted` contiene el valor cifrado, y la clave maestra que lo descifra vive fuera de la base de datos (documento **05**, apartado 8).

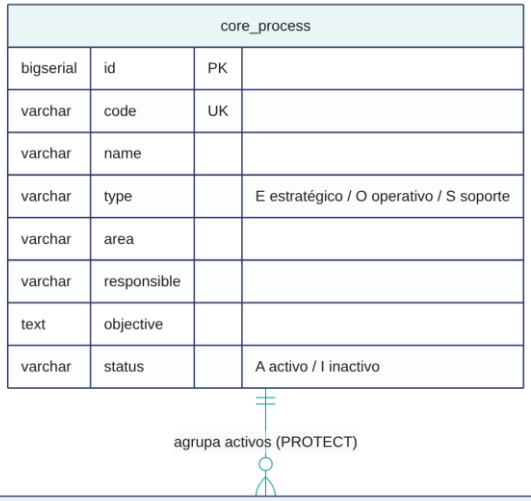
3. Gestión Documental



Notas de lectura

- **El archivo no vive en `core_document`**, vive en `core_documentversion`. Un documento es la ficha; cada archivo subido crea una versión numerada, y `current_version` es una copia del número de la última, mantenida por el sistema para no tener que consultar las versiones en cada listado.
- Las versiones son **evidencia**: no se editan ni se eliminan individualmente.
- El estado `V` (Vigente) **no se puede escribir directamente**: solo se alcanza mediante la acción de aprobación, que además registra quién aprobó y cuándo. Gestión Documental es el **único módulo del sistema con flujo de aprobación**.
- Las dos relaciones muchos a muchos se materializan en tablas puente generadas automáticamente: `core_document_related_documents` y `core_document_gap_requirements`.

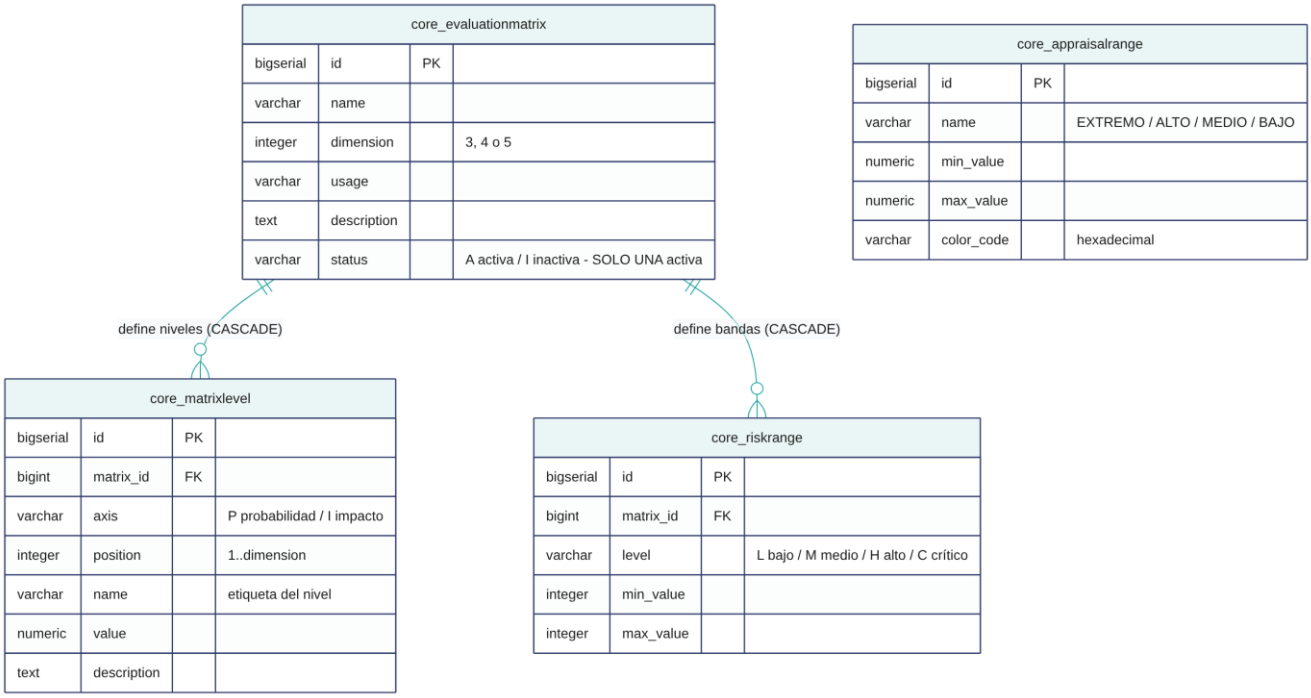
4. Procesos e Inventario de Activos



Notas de lectura

- `asset_value` **no se captura, se calcula** cada vez que se graba el activo, a partir de la confidencialidad, integridad y disponibilidad, con una de tres fórmulas seleccionable por activo (`cid_method`).
- `relevance` también es calculado, y solo se evalúa si el interruptor FONAFE de la empresa está encendido (ver diagrama 13).
- `core_asset` contiene **datos personales en sentido amplio** (finalidad, categorías de datos, base legal, plazo de conservación, flujos transfronterizos): es el registro de actividades de tratamiento exigido por la normativa de privacidad.
- La relación con `core_process` es `PROTECT` **y opcional**: un activo puede no tener proceso, pero un proceso con activos vivos no se puede eliminar.

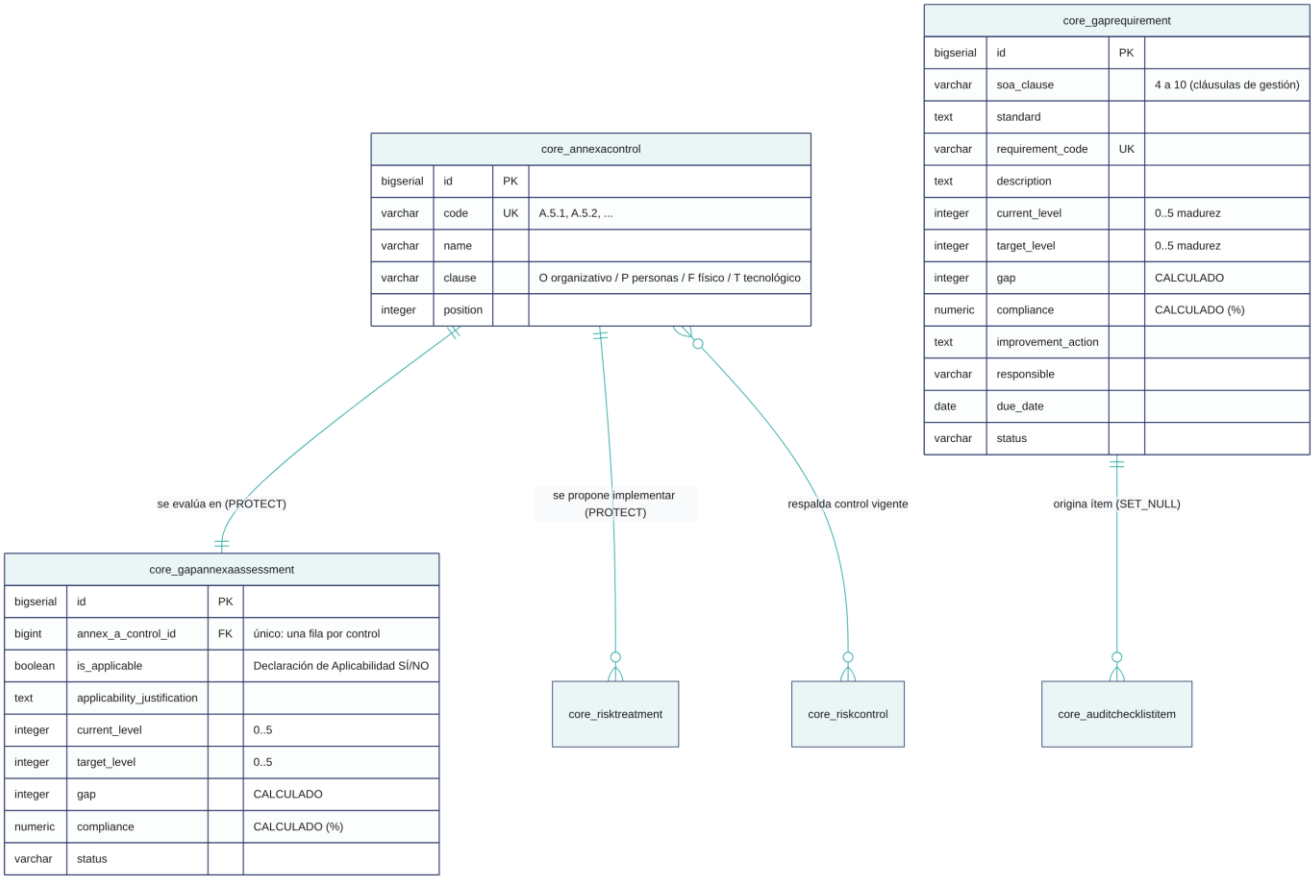
5. Metodología — matriz de evaluación y franjas de tasación



Notas de lectura

- Toda empresa nace con **tres matrices predefinidas** (3×3, 4×4 y 5×5) y con la 4×4 activa. Se editan y se activan, pero no se crean ni se eliminan.
- **Solo una matriz puede estar activa**, y eso lo garantiza la base de datos con una restricción única parcial, no una comprobación del programa (ver documento 03).
- La `dimension` de la matriz activa define la **escala de captura de todo el módulo de Riesgos** (1..N). No está fijada en el código.
- `core_riskrange` traduce un puntaje numérico a una severidad (Bajo/Medio/Alto/Crítico).
- `core_appraisalrange` **no está relacionada por llave foránea con ninguna otra tabla**: el cruce con `core_asset.asset_value` se resuelve en cada consulta comparando el valor del activo contra los intervalos. Cambiar una franja reclasifica todos los activos al instante, sin migrar datos.

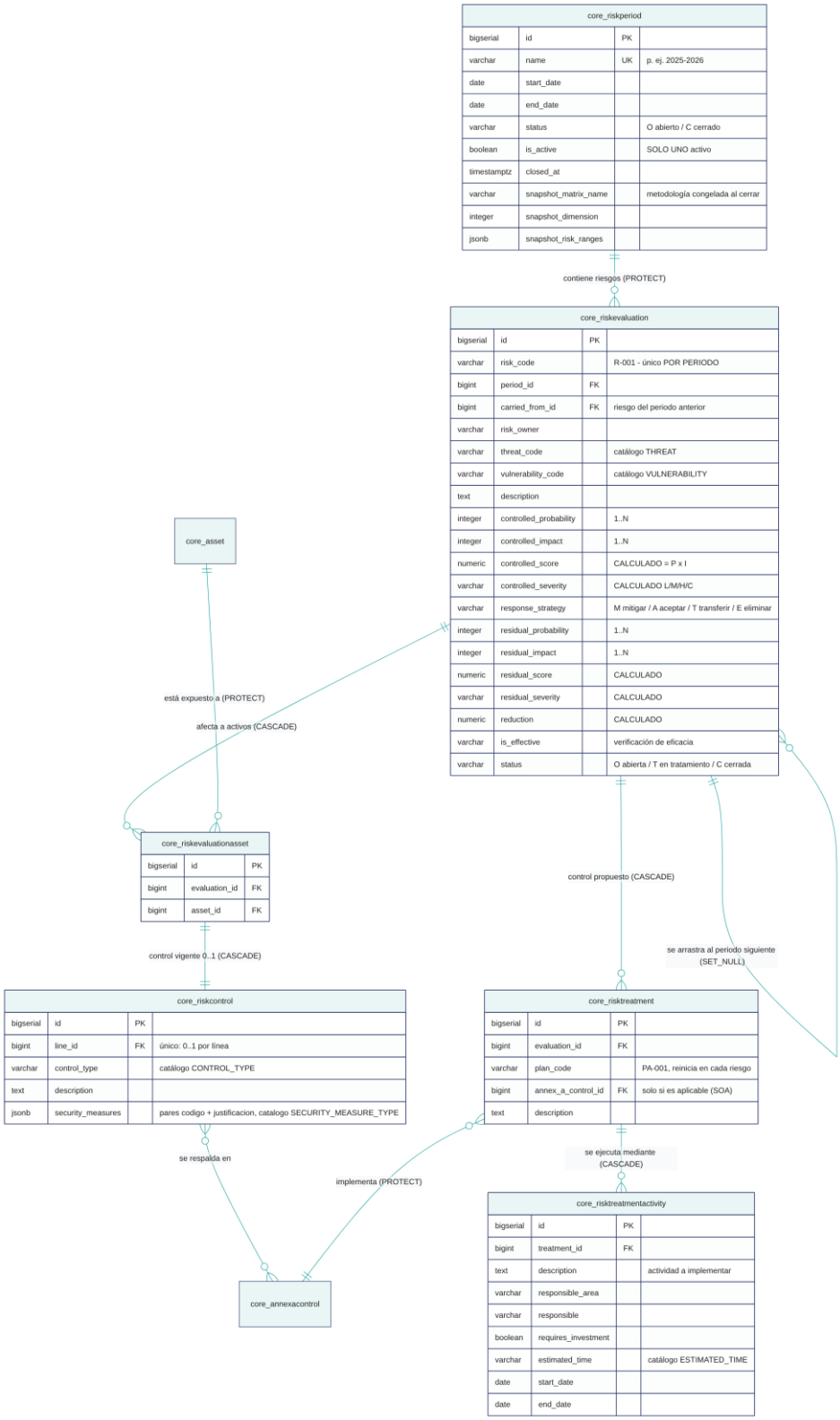
6. Matriz de Brecha y Anexo A (Declaración de Aplicabilidad)



Notas de lectura

- El módulo tiene **dos mitades con estructura distinta**:
  - `core_gaprequirement` cubre las **cláusulas de gestión 4 a 10** de la norma. Sus filas las escribe la empresa a mano, en texto libre.
  - `core_gapannexaassessment` cubre el **Anexo A**. Es diferente: tiene una llave foránea real a `core_annexacontrol` y sus **93 filas se crean solas** al dar de alta la empresa, una por cada control del catálogo. No se crean ni se eliminan, solo se editan.
- `core_annexacontrol` es un **catálogo de sistema inmutable**: los 93 controles del Anexo A de la norma ISO/IEC 27001:2022, sembrados automáticamente en cada empresa. Ninguna pantalla permite modificarlos.
- `core_gapannexaassessment.is_applicable` **es la Declaración de Aplicabilidad (SOA)** y tiene un efecto real en otro módulo: solo los controles marcados como aplicables pueden proponerse como tratamiento de un riesgo (diagrama 7).

7. Riesgos (DPIA / DPIIA) — el dominio más extenso



Notas de lectura — este dominio requiere cuatro aclaraciones

1. **El riesgo es reutilizable, la línea es lo que lo ancla a un activo.** `core_riskevaluation` **es el riesgo** (amenaza, vulnerabilidad, descripción, impacto y probabilidad **propios**). `core_riskevaluationasset` es **la línea activo–riesgo**: la fila que dice «este riesgo afecta a este activo». El mismo riesgo puede afectar a diez activos, y sus números son los mismos en los diez. Editar el riesgo se propaga a todos.

> Los nombres físicos de estas tablas no coinciden literalmente con los conceptos de negocio; su significado es el descrito aquí.

2. **El control vigente y el control propuesto son cosas distintas, y cuelgan de sitios distintos.**
    - El **control vigente** (`core_riskcontrol`) es lo que la empresa **ya tiene implementado** para ese activo concreto, y por eso cuelga de la **línea**: dos activos expuestos al mismo riesgo pueden tener controles vigentes diferentes. Es **como máximo uno por línea**, y esa cardinalidad la garantiza la base de datos mediante una relación uno a uno, no una convención del programa.
    - El **control propuesto** (`core_risktreatment`) es lo que la empresa **planea implementar**, se decide para el riesgo completo y por eso cuelga del riesgo. Cada control propuesto se ejecuta mediante **cero o más actividades** (`core_risktreatmentactivity`), cada una con su responsable, su plazo y sus fechas.
    - Asimetría deliberada con el Anexo A: el control **vigente** puede respaldarse en **varios** controles de la norma y **sin filtrar** por aplicabilidad (algo ya implementado es un hecho, aunque después se declare «no aplica»); el control **propuesto** apunta a **uno solo** y **debe** estar declarado aplicable en la Declaración de Aplicabilidad.
3. **Los riesgos se evalúan por temporadas.** Todo riesgo pertenece a un `core_riskperiod`. Por eso `risk_code` **es único por periodo, no globalmente**: el mismo `R-030` en 2025-2026 y en 2026-2027 es el **mismo riesgo seguido en el tiempo**, y `carried_from_id` los encadena. Un periodo cerrado queda en **solo lectura** y conserva copiada la metodología con la que se calculó, para que un cambio posterior de matriz no reescriba la historia.
4. **Los puntajes son calculados.** `controlled_score`, `residual_score`, `reduction` y las dos severidades los escribe el sistema al grabar; no se capturan ni se pueden enviar desde fuera.

8. Consentimientos de titulares

core_consent			
bigserial	id	PK	
varchar	consent_code	UK	C-001
varchar	data_subject_id		documento del titular - DATO PERSONAL
varchar	data_subject_name		DATO PERSONAL
varchar	email		DATO PERSONAL
varchar	phone		DATO PERSONAL
text	purpose		finalidad
varchar	legal_basis		
varchar	privacy_notice_version		
varchar	capture_method		W web / D documento / P presencial
timestamptz	granted_at		
timestamptz	expires_at		el estado Expirado se DERIVA de aquí
varchar	evidence_type		
varchar	evidence_file		PDF protegido
varchar	evidence_hash		SHA-256 calculado por el sistema
inet	ip_address		
varchar	status		O otorgado / R revocado
timestamptz	revoked_at		
text	revoked_reason		
integer	revoked_by_id	FK	

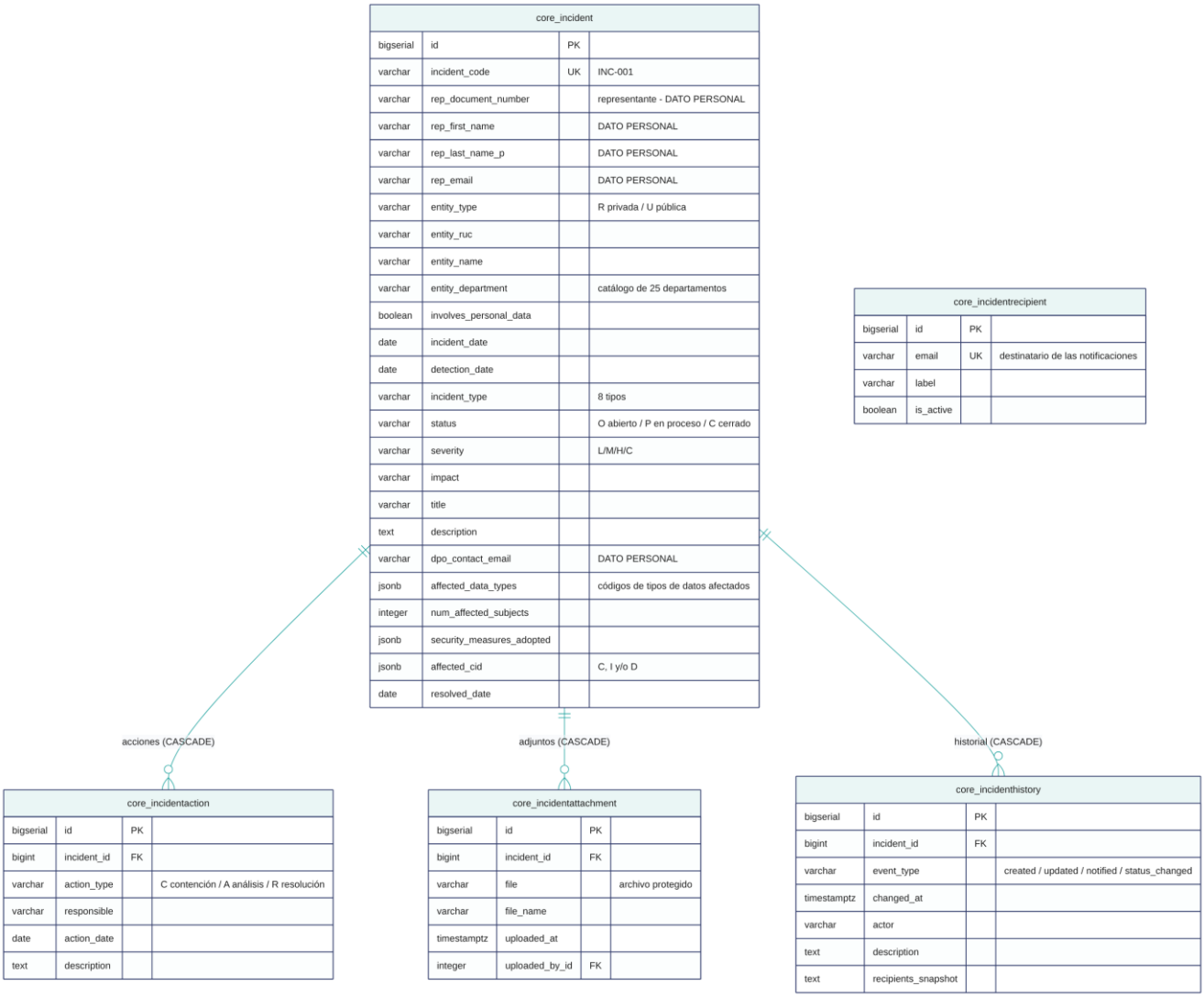
registra eventos (CASCADE)

core_consenthistory		
bigserial	id	PK
bigint	consent_id	FK
varchar	status	
timestamptz	changed_at	
varchar	actor	
integer	changed_by_id	FK
text	reason	
text	note	

Notas de lectura

- La columna `status` solo tiene **dos valores**: Otorgado y Revocado. El estado **«Expirado» no se guarda**: se deriva comparando `expires_at` con la fecha actual cada vez que se consulta. No hay proceso programado que actualice estados.
- `evidence_hash` es la huella SHA-256 del PDF, calculada por el sistema al subirlo. Sirve para demostrar ante un auditor que la evidencia no fue alterada después.
- `core_consenthistory` es una **tabla inmutable**: solo admite inserciones. Modificarla o borrar una fila está bloqueado a nivel de modelo (ver documento 04).
- Esta tabla contiene **datos personales de terceros** (los titulares), no de empleados. Ver documento 05.

9. Incidentes de seguridad y privacidad

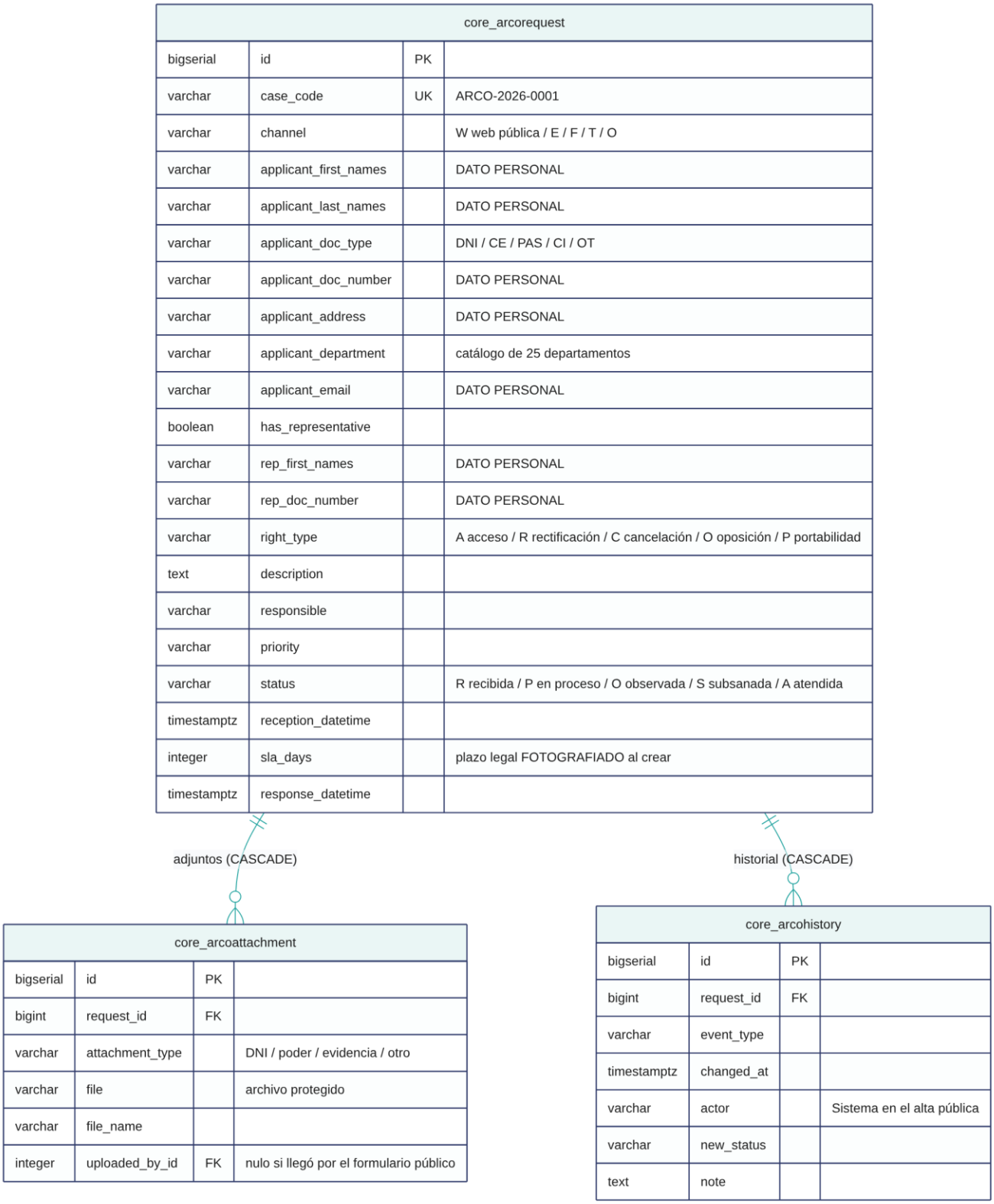


Notas de lectura

- `core_incident` reproduce el **formulario oficial de reporte de incidentes** de la autoridad peruana de protección de datos. Por eso lleva embebidos los datos del representante y de la entidad que reporta: son campos del formulario, no entidades del sistema.

- Las selecciones múltiples (`affected_data_types`, `security_measures_adopted`, `affected_cid`) se guardan como **listas JSON de códigos** contra catálogos fijos, en lugar de crear tablas adicionales.
- `core_incidentattachment` es el **primer y único caso del sistema en que una entidad admite varios archivos**.
- `core_incidenthistory` es **inmutable**, igual que el historial de consentimientos.
- `core_incidentrecipient` **no cuelga de ningún incidente**: es una lista de correos a nivel de empresa, compartida por todos los incidentes, que se usa al notificar.
- Es la tabla del sistema con **mayor densidad de datos personales**. Ver documento 05.

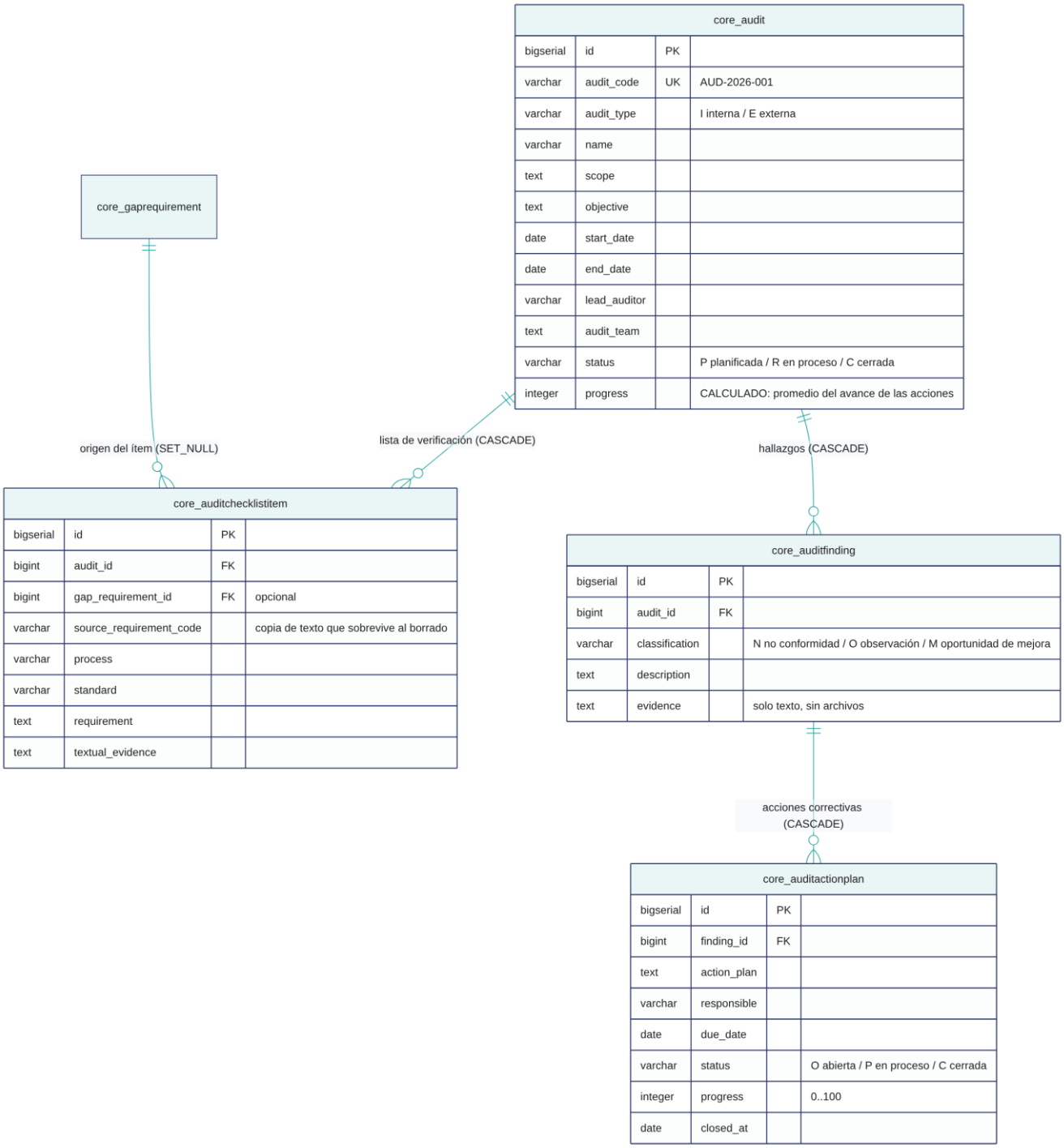
10. Solicitudes ARCO-P (Mesa de Partes Digital)



Notas de lectura

- Este es el **único módulo con una superficie pública**: un formulario sin inicio de sesión por el que un ciudadano ejerce sus derechos. Ese formulario **solo puede crear** solicitudes; nunca listar ni leer las de terceros. Ver documento 05.
- `sla_days` es una **fotografía**: al crear el caso se copia el plazo legal que corresponde al derecho solicitado (20 días calendario para Acceso, 10 para los demás) y **no se recalcula** aunque después cambie el catálogo. La fecha límite, los días restantes y el semáforo de vencimiento **se derivan en cada consulta** de `reception_datetime + sla_days`; no hay columnas ni procesos programados para eso.
- `created_by_id` puede ser **nulo**, a diferencia de casi todas las demás tablas: cuando la solicitud entra por el formulario público no hay ningún usuario del sistema ejecutando la operación.
- `core_arcohistory` es **inmutable**.

11. Auditorías

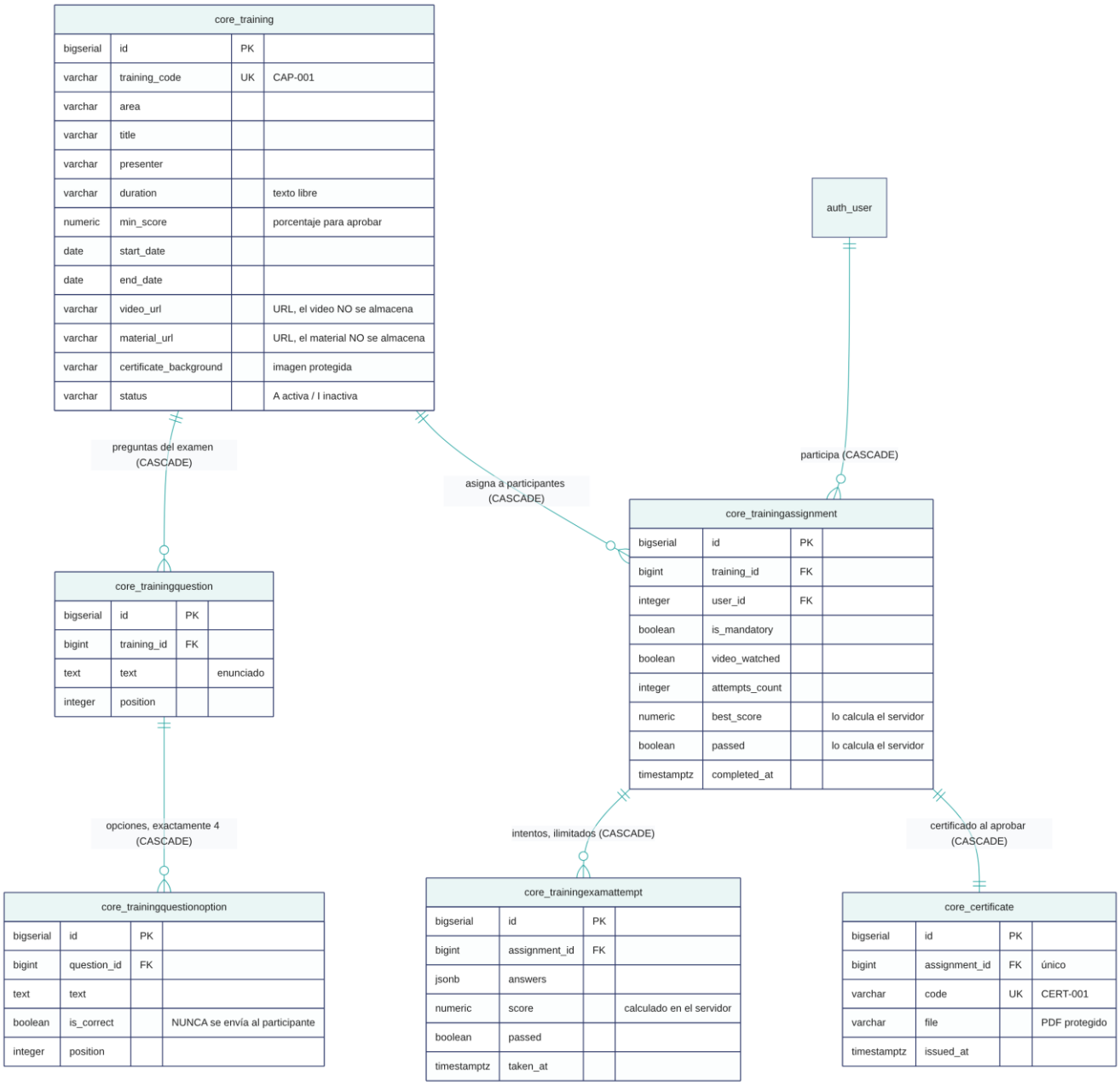


Notas de lectura

- `core_auditchecklistitem` es un **duplicado editable a propósito** de un requisito de la Matriz de Brecha. La llave foránea es `SET_NULL` y, además, el código del requisito se copia como texto en `source_requirement_code`: si mañana alguien borra el requisito original, la auditoría **conserva la trazabilidad de qué se auditó**. Una auditoría cerrada es evidencia y no puede quedar vacía por un cambio posterior.

- `core_audit.progress` es **calculado**: promedio del avance de todas las acciones de todos los hallazgos. No se captura.
- El vencimiento de una acción correctiva **se deriva** comparando `due_date` con la fecha actual; no hay columna ni proceso programado.

12. Capacitaciones

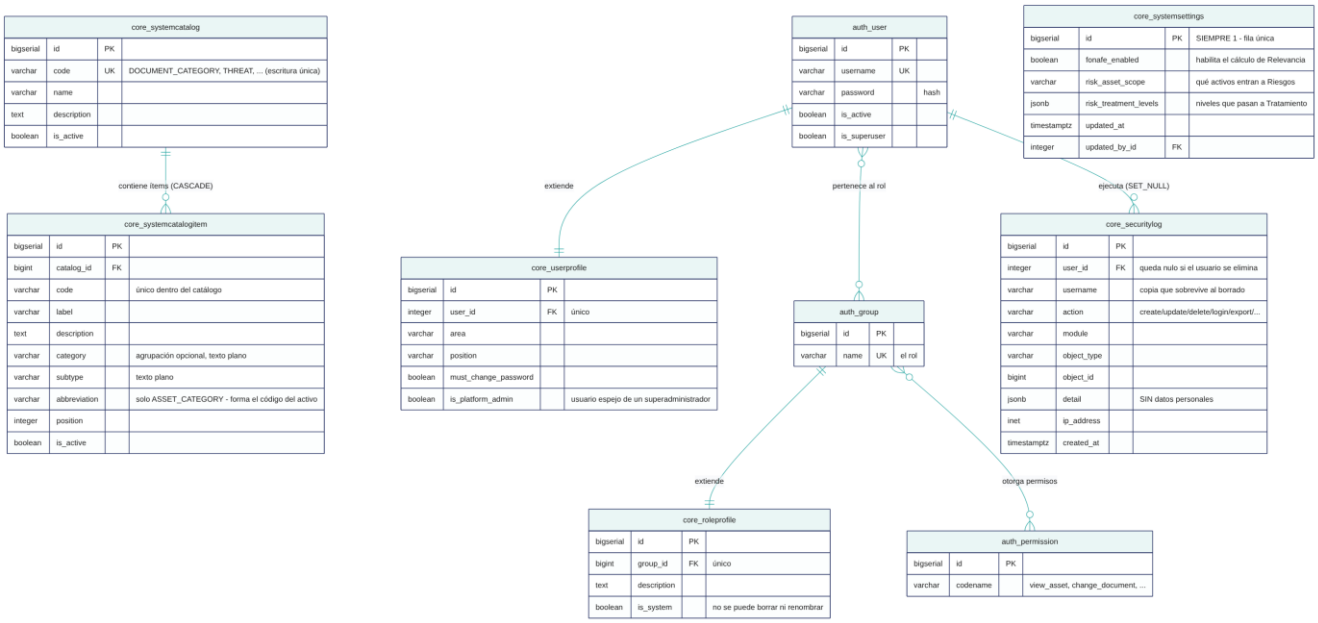


Notas de lectura

- `core_trainingquestionoption.is_correct` es la columna más sensible del módulo: **nunca viaja al participante**. La corrección del examen ocurre íntegramente en el servidor; el cliente envía respuestas y recibe un puntaje, sin posibilidad de conocer ni falsificar el resultado.

- `core_trainingexamattempt` es **solo de adición**: cada intento deja su fila. Los reintentos son ilimitados y el histórico completo queda disponible.
- El certificado se genera **una sola vez**, al aprobar, y se guarda como PDF. La relación con la asignación es uno a uno.
- El video y el material de apoyo **no se almacenan en el sistema**: se guardan sus URL. El único archivo que el módulo guarda es la imagen de fondo del certificado.
- `core_trainingassignment.user_id` apunta a la tabla `auth_user` **del esquema de esa empresa**, no a la del esquema `public`.

13. Configuración del Sistema, identidad y bitácora



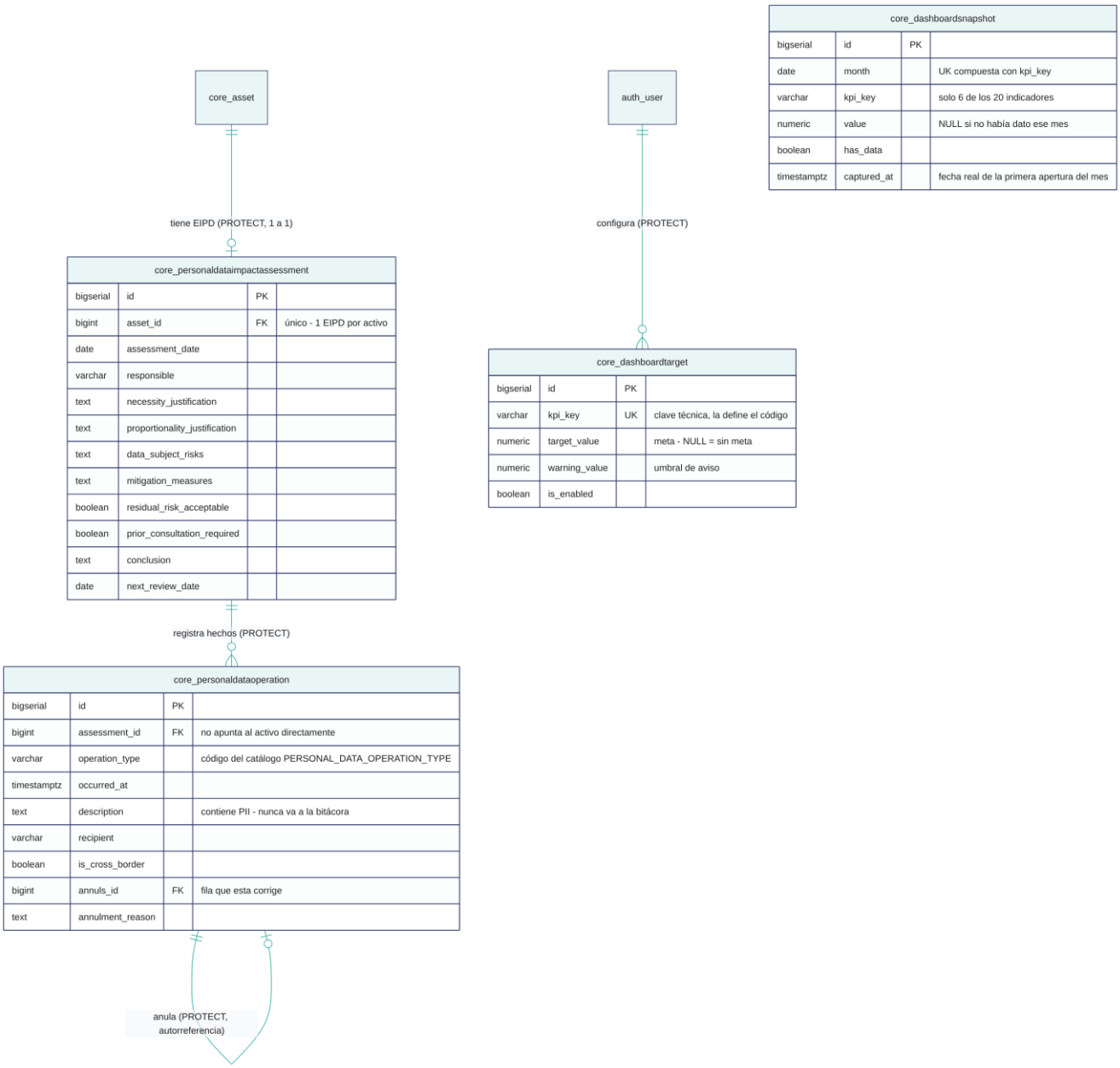
Notas de lectura

- `core_systemcatalog` / `core_systemcatalogitem` **son la infraestructura que evita listas de valores fijas en el código.** Siete catálogos usan hoy este mecanismo: categorías de documento, categoría / clasificación / frecuencia de uso de activo, tipo de control, amenazas y vulnerabilidades, además de tipo de medida de seguridad y tiempo estimado. Los módulos guardan **el código del ítem como texto**, no una llave foránea; eso permite que un registro histórico conserve su valor aunque el ítem se desactive después. Los catálogos de **amenazas y vulnerabilidades** merecen una nota: se entregan con una **lista de arranque editable** (57 amenazas y 82 vulnerabilidades), **no normativa**. La norma ISO/IEC 27005 retiró en su edición de 2022 los anexos que listaban unas y otras, de modo que no existe una lista oficial vigente que imponer; cada empresa ajusta o reemplaza la suya por completo desde Configuración del Sistema, importándola en Excel.
- `core_systemsettings` **es una tabla de una sola fila.** Su llave primaria se fuerza siempre a 1 y su borrado está bloqueado: la fila no puede desaparecer, aunque su contenido sí se edita. Es donde viven los interruptores que afectan a toda la empresa.

- `core_userprofile` y `core_roleprofile` **extienden** las tablas estándar de Django sin reemplazarlas: los usuarios siguen en `auth_user` y los roles en `auth_group`. Toda la autorización se apoya en `auth_permission`, que es el mecanismo nativo de Django.
- `core_securitylog` es la **bitácora inmutable**: solo admite inserciones, y tanto la modificación como el borrado están bloqueados a nivel de modelo. Es la única tabla del sistema que **ningún administrador puede alterar**. Su columna `detail` nunca contiene datos personales ni valores de campos, solo la referencia corta del objeto y los nombres de los campos tocados.

14. Dashboard Ejecutivo y EIPD (Datos Personales)

(Dos pares de tablas sin relación entre sí: se agrupan en un solo diagrama por ser pequeños, no porque compartan datos.)



Notas de lectura

- **El Dashboard Ejecutivo no tiene tabla de relaciones con el resto del sistema.** `core_dashboardtarget` (20 filas, una por indicador) y `core_dashboardsnapshot` (una foto mensual de 6 series) son las únicas dos tablas que persiste; los 20 indicadores y las 9 gráficas que muestra la pantalla de inicio se **calculan en el momento**, leyendo de las tablas que ya existen en los otros 13 diagramas — nunca se duplica un dato.
- **La EIPD se ancla al activo, no al riesgo.** Un activo marcado como «de datos personales» en el Inventario tiene como máximo una EIPD (relación uno a uno, forzada por la base de datos), que se crea sola en cuanto el activo se marca así — no hace falta un paso manual. Es independiente del módulo de Riesgos: un activo sigue necesitando su EIPD aunque hoy no califique para la bandeja de Riesgos por su tasación o por el periodo de evaluación activo.
- **El registro de operaciones es una bitácora de hechos, inmutable con corrección por anulación**, igual que `core_consenthistory`/`core_incidenthistory` del resto del sistema, pero con un mecanismo que esos

dos no tienen: una fila puede **anular** a otra anterior (`annuls_id` + motivo obligatorio) sin editarla ni borrarla, dejando ambas visibles.

- `core_personaldataoperation.description` **contiene datos personales de terceros** (a quién se compartió información, con qué motivo); a diferencia del resto de operaciones del sistema, este texto **nunca** se vuelca a `core_securitylog`.

Resumen de cardinalidades singulares

Vale la pena tener presentes estos seis casos, que se salen de la relación uno-a-muchos habitual y explican decisiones de diseño:

Relación	Cardinalidad	Por qué
`core_riskevaluationasset` → `core_riskcontrol`	**0..1**, garantizada por la base de datos	Una línea activo–riesgo declara como máximo un control vigente. La restricción es física (relación uno a uno), no una comprobación del programa.
`core_annexacontrol` → `core_gapannexaassessment`	**1 a 1**, provisionada automáticamente	Existe exactamente una evaluación por cada uno de los 93 controles del Anexo A, creada al dar de alta la empresa.
`core_asset` → `core_personaldataimpactassessment`	**0..1**, garantizada por la base de datos y creada sola	Un activo de datos personales tiene como máximo una EIPD; se crea automáticamente al marcar el activo como tal, sin paso manual (diagrama 14).
`core_trainingassignment` → `core_certificate`	**0..1**	Un certificado se emite una sola vez, al aprobar.
`core_trainingquestion` → `core_trainingquestionoption`	**exactamente 4, con 1 correcta**	La cardinalidad exacta la impone la capa de validación, no la base de datos.
`core_systemsettings`	**exactamente 1 fila**	Tabla de ajustes de empresa con llave primaria forzada a 1 y borrado bloqueado.

Documentos relacionados

Documento	Contenido
02 — Diccionario de datos	Las 52 tablas columna por columna, con tipo, nulabilidad y significado.
03 — Esquema de la base de datos	Llaves primarias y foráneas, restricciones, índices y reglas de integridad.
04 — Procedimientos SQL y scripts	Dónde vive realmente la lógica de negocio y qué scripts operativos existen.
05 — Políticas de acceso y seguridad	Quién puede ver, modificar o borrar cada dato.